

Basics

textual literals

In [1]:

```
print('hello')
```

hello

User Input

In []:

```
number = eval(input("Enter an expression:"))  
number
```

Datatypes

In [4]:

```
float(5)
```

Out[4]:

5.0

In [5]:

```
int(5.64687)
```

Out[5]:

5

In [6]:

```
str(55.2210)
```

Out[6]:

'55.221'

In [7]:

```
str(10.00)
```

Out[7]:

'10.0'

In [8]:

```
str(10.0)
```

Out[8]:

'10.0'

In [9]:

```
str(10.06)
```

Out[9]:

'10.06'

In [10]:

```
str(10.654)
```

Out[10]:

```
'10.654'
```

```
In [11]:
```

```
_var = 3  
_var
```

```
Out[11]:
```

```
3
```

```
In [1]:
```

```
user_input = eval(input("Enter any expression:"))  
user_input
```

```
Out[1]:
```

```
20.7
```

```
In [4]:
```

```
name = "shawaiz"  
age = 20  
str(age)
```

```
Out[4]:
```

```
'20'
```

```
In [5]:
```

```
float(age)
```

```
Out[5]:
```

```
20.0
```

```
In [7]:
```

```
x,y,z = 5,8.4,900  
print(x) # numerical literals  
print(y)  
print(z)
```

```
5
```

```
8.4
```

```
900
```

```
In [4]:
```

```
#BOOLEAN
```

```
isRunning = False  
isRunning
```

```
Out[4]:
```

```
False
```

```
In [11]:
```

```
x = abs(-92)  
x
```

```
Out[11]:
```

```
92
```

```
In [16]:
```

```
model = int("57")  
model
```

```
Out[16]:
```

```
57
```

```
In [20]:
```

```
model = 2 + 3.2 # implicit type conversion
model
```

Out[20]:
5.2

In [23]:

```
height = 8 // 2 + 2.6
height
```

Out[23]:
6.6

In [24]:

```
"""
This is a multi-line comment.
Using triple quotes (single or double).
"""
```

Out[24]:
'\nThis is a multi-line comment.\nUsing triple quotes (single or double).\n'

In [25]:

```
animals = ["zebra", "deer", "lion", "cheeta", "elephant", "rhino", "leopard", "horse"]
animals
```

Out[25]:
['zebra', 'deer', 'lion', 'cheeta', 'elephant', 'rhino', 'leopard', 'horse']

In [36]:

```
for i in animals:
    print(i)
```

zebra
deer
lion
cheeta
elephant
rhino
leopard
horse

In [29]:

```
print??
```

Signature: `print(*args, sep=' ', end='\n', file=None, flush=False)`

Docstring:

Prints the values to a stream, or to sys.stdout by default.

sep

string inserted between values, default a space.

end

string appended after the last value, default a newline.

file

a file-like object (stream); defaults to the current sys.stdout.

flush

whether to forcibly flush the stream.

Type: builtin_function_or_method

In [34]:

```
print?
```

Signature: `print(*args, sep=' ', end='\n', file=None, flush=False)`

Docstring:

Prints the values to a stream, or to `sys.stdout` by default.

`sep`

string inserted between values, default a space.

`end`

string appended after the last value, default a newline.

`file`

a file-like object (stream); defaults to the current `sys.stdout`.

`flush`

whether to forcibly flush the stream.

Type: `builtin_function_or_method`

In [37]:

```
for i in animals:
    print(i, end=", ")
```

zeebra, deer, lion, cheeta, elephant, rhino, leopard, horse,

In [54]:

```
for index, item in enumerate(animals):
    print(f"{index}: {item}") # Using f-string
```

0: zeebra
1: deer
2: lion
3: cheeta
4: elephant
5: rhino
6: leopard
7: horse

In [47]:

```
help("for")
```

The "for" statement

The "for" statement is used to iterate over the elements of a sequence (such as a string, tuple or list) or other iterable object:

```
for_stmt ::= "for" target_list "in" starred_list ":" suite
          ["else" ":" suite]
```

The "starred_list" expression is evaluated once; it should yield an *iterable* object. An *iterator* is created for that iterable. The first item provided by the iterator is then assigned to the target list using the standard rules for assignments (see Assignment statements), and the suite is executed. This repeats for each item provided by the iterator. When the iterator is exhausted, the suite in the "else" clause, if present, is executed, and the loop terminates.

A "break" statement executed in the first suite terminates the loop without executing the "else" clause's suite. A "continue" statement executed in the first suite skips the rest of the suite and continues with the next item, or with the "else" clause if there is no next item.

The for-loop makes assignments to the variables in the target list. This overwrites all previous assignments to those variables including those made in the suite of the for-loop:

```
for i in range(10):
    print(i)
    i = 5           # this will not affect the for-loop
                   # because i will be overwritten with the next
                   # index in the range
```

Names in the target list are not deleted when the loop is finished, but if the sequence is empty, they will not have been assigned to at all by the loop. Hint: the built-in type "range()" represents immutable arithmetic sequences of integers. For instance, iterating "range(3)" successively yields 0, 1, and then 2.

Changed in version 3.11: Starred elements are now allowed in the expression list.

Related help topics: [break](#), [continue](#), [while](#)

In [56]:

```
range??
```

Init signature: range(self, /, *args, **kwargs)

Docstring:

range(stop) -> range object

range(start, stop[, step]) -> range object

Return an object that produces a sequence of integers from start (inclusive) to stop (exclusive) by step. range(i, j) produces i, i+1, i+2, ..., j-1. start defaults to 0, and stop is omitted! range(4) produces 0, 1, 2, 3. These are exactly the valid indices for a list of 4 elements. When step is given, it specifies the increment (or decrement).

Type: type

Subclasses:

In [71]:

```
for i in range(0, 20, 5): # range(start, stop, increment)
    print(i)
```

```
0
5
10
15
```

In [73]:

```
for i in range(20, 0, -5): # range(start, stop, decrement)
    print(i)
```

```
20
15
10
5
```

In [48]:

```
for index in range(len(animals)):
    item = animals[index]
    print(f"{index}: {item}")
```

```
0: zebra
1: deer
2: lion
3: cheeta
4: elephant
5: rhino
6: leopard
7: horse
```

In [49]:

```
enumerate??
```

Init signature: enumerate(iterable, start=0)

Docstring:

Return an enumerate object.

iterable
an object supporting iteration

The enumerate object yields pairs containing a count (from start, which defaults to zero) and a value yielded by the iterable argument.

enumerate is useful for obtaining an indexed list:

(0, seq[0]), (1, seq[1]), (2, seq[2]), ...

Type: type

Subclasses:

In [53]:

```
for i in enumerate(animals, start=1):
    print(i)
```

```
(1, 'zebra')
(2, 'deer')
(3, 'lion')
(4, 'cheeta')
(5, 'elephant')
(6, 'rhino')
(7, 'leopard')
(8, 'horse')
```

In [55]:

```
for index, item in enumerate(animals, start=1):
    print(f"{index}: {item}")
```

```
1: zebra
2: deer
3: lion
4: cheeta
5: elephant
6: rhino
7: leopard
8: horse
```

In [82]:

```
i = 0
while i < len(animals):
    print(f"{i+1}-{animals[i]}", end=", ")
    i += 1
```

1-zebra, 2-deer, 3-lion, 4-cheeta, 5-elephant, 6-rhino, 7-leopard, 8-horse,

In [89]:

```
def func():  
    print("hello")  
  
func()
```

hello

In [94]:

```
def func(*argv): # for multiple arguments  
    for i in argv:  
        print(i)  
  
func("animals", 5, 6.387, 50+65, 0)
```

animals
5
6.387
115
0

In []:

```
"""  
Reusable programs are usually created by typing definitions into a separate file  
called a module or script.  
■ This file is saved on disk so that it can be used over and over again  
  
■ A Python module file is just a text file with a .py extension, which can be created us  
any program for editing text (e.g., notepad or vim)  
  
■ Namespace Isolation: Each module defines its own namespace, preventing naming conflict  
identifiers (variables, functions, classes) in different parts of a program.  
  
■ Built-in Modules: math for mathematical operations, random for generating random numbe  
and os for interacting with the operating system.
```

path/module.py

```
-----  
def printMsg(msg):  
    print (msg*2)  
-----
```

Notebook or any other python program

```
-----  
import sys  
sys.path.append('path to folder where module.py exists')  
import module as myf
```

```
myf.printMsg("hello")  
-----  
  
"""
```

In []:

```
!pip install jupyter
```

In [42]:

```
import jupyter as jt
jt.make_turtle()
jt.forward(100)
```



In [47]:

```
from jupyter import make_turtle, forward, left, right
```

In [49]:

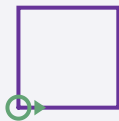
```
make_turtle()

forward(50)
left(90)

forward(50)
left(90)

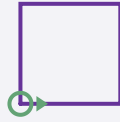
forward(50)
left(90)

forward(50)
left(90)
```



In [50]:

```
make_turtle()
for i in range(4):
    forward(50)
    left(90)
```



Data Structures

• List	Ordered,	changeable,	duplicates
• Tuple	Ordered,	unchangeable,	duplicates
• Set	Unordered,	addable/removable	no duplicates
• Dictionary	Unordered,	changeable,	no duplicate

List	Tuple	Set	Dictionary
<code>L = [12, "banana", 5.3]</code>	<code>T = (12, "banana", 5.3)</code>	<code>S = {12, "banana", 5.3}</code>	<code>D = {"Val": 12, "name": "Ban"}</code>
<code>L[1]</code>	<code>T[2]</code>	<code>X in S</code>	<code>D["Val"]</code>
<code>L = L + ["game"]</code> <code>L[2] = "orange"</code>	Immutable <code>T3 = T1 + T2</code>	<code>S.add("new Item")</code> <code>S.update({"more", "items"})</code>	<code>D["Val"] = newValue</code> <code>D["newkey"] = "newVal"</code>
<code>del L[1]</code> <code>del L</code>	Immutable <code>del T</code>	<code>S.Remove("banana")</code> <code>del S</code>	<code>del D["Val"]</code> <code>del D</code>
<code>L2 = L.copy()</code>	<code>T2 = T</code>	<code>S2 = S.copy()</code>	<code>D2 = D.copy()</code>
....			

Tuple

- **Ordered:** The elements in a tuple have a defined order, and this order will not change. Each element has an index, starting from 0.
- **Immutable:** This is the most significant characteristic. Once a tuple is created, its contents cannot be modified. You cannot change an element, add new elements, or remove existing ones.

Sets

- **Unordered:** Elements within a set do not have a defined order, and their position can change. This means you cannot access elements by index.
- **Unique Elements:** Sets automatically handle duplicate entries, ensuring that each element appears only once. If you attempt to add a duplicate, it will simply be ignored.
- **Immutable Elements:** While sets themselves are mutable (elements can be added or removed), the elements they contain must be immutable data types (e.g., numbers, strings, tuples). Mutable types like lists or dictionaries cannot be direct elements of a set.

- **Mutable:** You can add or remove elements from a set after it has been created.

In [15]:

```
arr = ['test', 13, 1.131]
arr += ["game"]
arr
```

Out[15]:

```
['test', 13, 1.131, 'game']
```

In [13]:

```
del arr[0]
arr
```

Out[13]:

```
[13, 1.131, 'game']
```

In [16]:

```
arr2 = arr.copy()
arr2
```

Out[16]:

```
['test', 13, 1.131, 'game']
```

In [17]:

```
del arr2
```

```
arr2
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[17], line 3
      1 del arr2
----> 3 arr2

NameError: name 'arr2' is not defined
```

In [18]:

```
# An empty tuple
empty_tuple = ()

# A tuple with mixed data types
my_tuple = (1, "hello", 3.14, True)

# A tuple with a single element (note the comma)
single_element_tuple = ("apple",)
print(single_element_tuple, len(single_element_tuple))

# Creating a tuple from an iterable using the tuple() constructor
my_list = [1, 2, 3]
tuple_from_list = tuple(my_list)
```

```
('apple',) 1
```

In [18]:

```
my_tuple = (10, 20, 30, 40)

print(my_tuple[0]) # Output: 10
print(my_tuple[-1]) # Output: 40 (last element)
print(my_tuple[1:3]) # Output: (20, 30) (slicing)
```

```
tuple_copy = my_tuple
print(tuple_copy)
```

```
10
40
(20, 30)
(10, 20, 30, 40)
```

In [30]:

```
my_set = {1, 2, 3, "apple", "banana"}
my_set
```

Out[30]:

```
{1, 2, 3, 'apple', 'banana'}
```

In [26]:

```
another_set = set([1, 2, 2, 3, "orange"]) # Duplicates are removed automatically
empty_set = set() # To create an empty set, use set(), not {} (which creates an empty di
another_set
```

Out[26]:

```
{1, 2, 3, 'orange'}
```

In [27]:

```
my_set.add(4)
my_set.update([5, 6]) # Add multiple elements from an iterable
my_set
```

Out[27]:

```
{1, 2, 3, 4, 5, 6, 'apple', 'banana'}
```

In [28]:

```
my_set.remove(1) # Raises KeyError if element not found
print(my_set)
my_set.discard("apple") # Does not raise an error if element not found
print(my_set)
my_set.pop() # Removes and returns the first element
print(my_set)
my_set.clear() # Removes all elements
print(my_set)
```

```
{2, 3, 4, 5, 6, 'apple', 'banana'}
{2, 3, 4, 5, 6, 'banana'}
{3, 4, 5, 6, 'banana'}
set()
```

In [35]:

```
my_set = {"apple", "banana"}
print(my_set)

if "banana" in my_set:
    print("Banana is in the set.")
```

```
{'apple', 'banana'}
Banana is in the set.
```

In []:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}

union_set = set1.union(set2) # {1, 2, 3, 4, 5}
intersection_set = set1.intersection(set2) # {3}
```

```
difference_set = set1.difference(set2)    # {1, 2} (elements in set1 but not in set2)
symmetric_difference_set = set1.symmetric_difference(set2) # {1, 2, 4, 5} (elements in e
# set but not both)
```

In [36]:

```
for i in my_set:
    print(i)
```

apple
banana

In [37]:

```
set_copy = my_set.copy()
set_copy
```

Out[37]:

{'apple', 'banana'}